

LT12 – Quick Sort

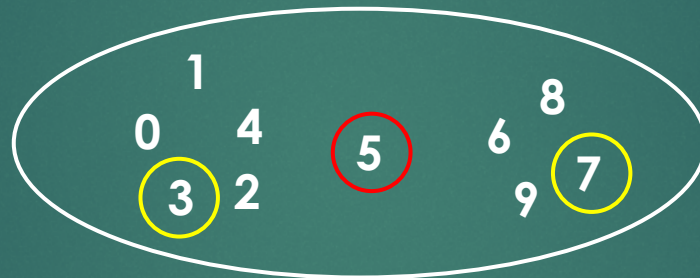
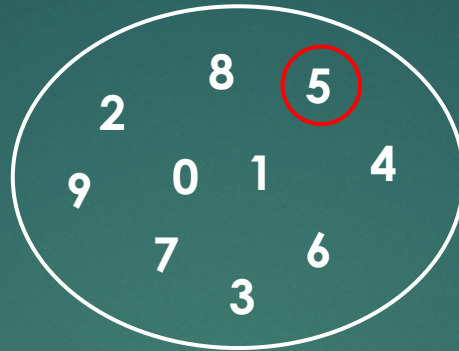
ALGORITHM

Unplugged Activity



1. Start with a stack of 10 cards showing all the numbers.
2. Pick a pivot and shift all the cards with smaller numbers on the left and larger numbers on the right of the pivot.
3. Repeat Step 2 for the left and the right halves.
4. Keep repeating until the cards are sorted.

Unplugged Activity



Description :

The **Quick Sort** is a 'Divide and Conquer' sorting algorithm.

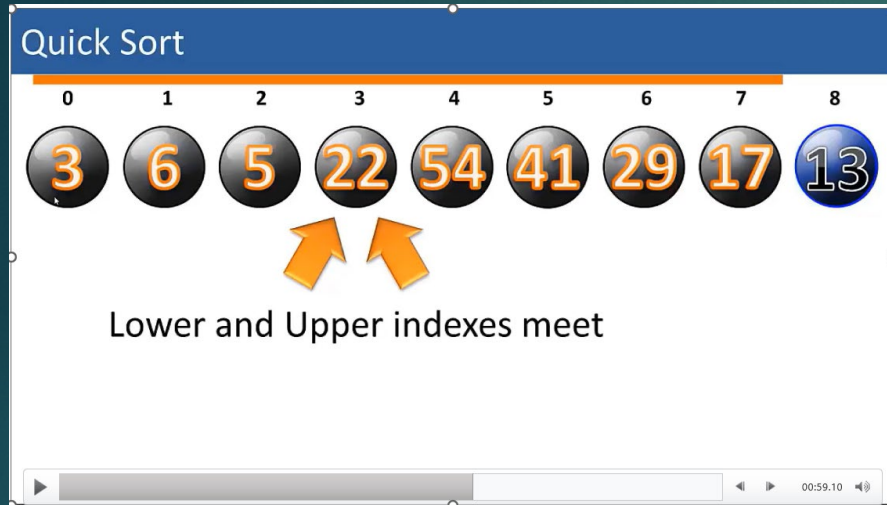
Pick a pivot to partition the array into two halves

- one side containing all the elements less than the pivot and the other side containing the elements greater than the pivot.

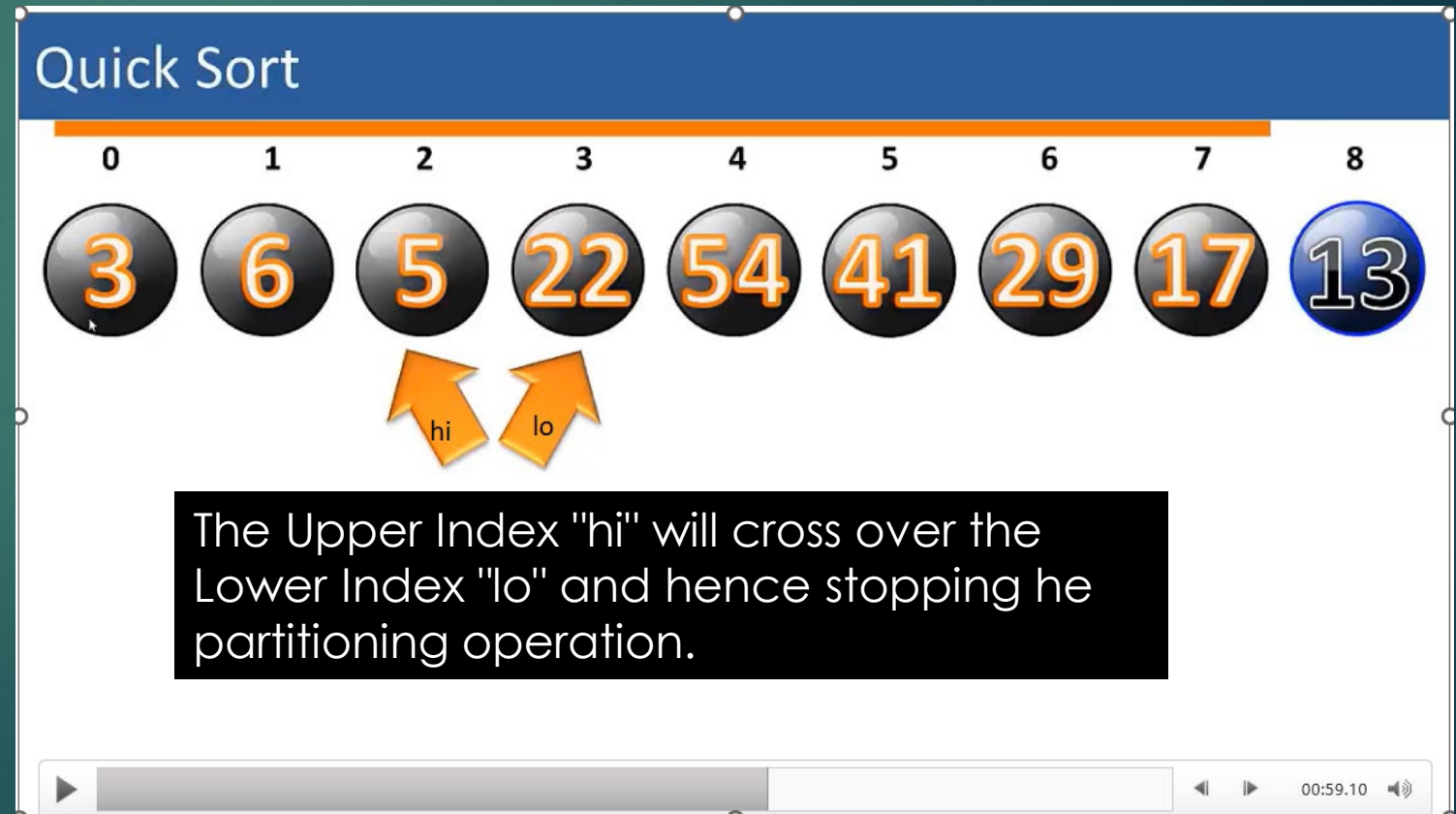
Repeat the same process with each halves of the array recursively to eventually obtain a sorted array.

Quick Sort Explained (Extract)

Video correction at 1:45-1:55



The Upper Index "hi" will continue to shift one step to the left since 22 is not smaller than the Pivot.



Step 1 : Repeatedly Partitioning a Sequence

- a. Select a Pivot and define the 'lo' and 'hi' pointers.
- b. Shift the 'lo' pointer to the right until an element larger than the pivot is found.
- c. Shift the 'hi' pointer to the left until an element smaller than the pivot is found.
- d. Swap the elements in the 'lo' and 'hi' pointer positions.
- e. Repeat (a) to (d) until the 'lo' pointer is larger than the 'hi' pointer.
- f. Swap the Pivot and the element at the 'lo' pointer.

Step 2 : Recursively partition the Sequence

Write the recursive program code for the function `qsort(seq,Start,End)` to call the *Helper function* `Partition()` written in Step 1.

Step 3 : Create a wrapper function

Write program code for the *Wrapper function* `quicksort(seq)`.

Quick Sort

Order of Growth

Worst Case : $O(n^2)$

~ where every pivot picked is either the largest or the smallest element the list.

Best Case : $O(n \log n)$

~ about equal trees after each partition

Average Case : $O(n \log n)$



The End

QUICKSORT ALGORITHM